SOFTWARE DESIGN PATTERNS (SDP) • FINAL PROJECT REPORT

Banking System Web Application

Design and implementation of a concurrent Go web service integrating six Gang of Four (GoF) design patterns with a single-page web client.

Team Members: Shynarbek Nuraziz (Student A), Asset Kamza (Student B)

Date: November 11, 2026

Tech Stack: Go 1.27, Echo v4, JSON Ledger Persistence, HTML/CSS/JS SPA

Repository: https://github.com/Qpzm1212/FInal-SDP-Banking-SyStem

1. Introduction & Project Scope

This project is a working banking web application written in Go using the Echo framework, paired with a single-page frontend. Rather than demonstrating Gang of Four (GoF) design patterns in isolated examples, the system connects six patterns inside one cohesive banking domain covering account policies, concurrent transfers with AML and velocity fraud monitoring, loan contract generation, and interest calculation.

All business rules, domain limits, and storage persistence live on the Go backend. The browser interface acts strictly as a client communicating over JSON REST endpoints.

- **Account Management:** Open checking, savings, and credit accounts with ID format validation, per-product transfer caps ($50k / $25k / $5k), and a $5,000 credit limit.
- **Transfers & Anti-Fraud Monitoring:** Apply debits safely under concurrency, log every operation to stdout, and flag transfers exceeding $10,000 or rapid velocity (>3 tx/min).
- **Loan Agreements:** Build validated 12-month loan contracts ($100 to $500,000) using the standard annuity formula.
- **Interest Calculator:** Compute annual yield (0% checking, 5% savings, 15% credit/loan) without conditional branching.

2. System Architecture & API

The codebase follows a layered layout where HTTP handlers delegate directly to a service facade, keeping transport concerns separate from domain patterns and file persistence.

Path	Layer	Responsibility
`cmd/api/main.go`	HTTP Transport	Echo server setup, static file serving, request parsing, error-to-HTTP status mapping.
`internal/service/facade.go`	Service Facade	`BankFacade` orchestrates patterns, account limits, and exposes banking operations.
`internal/patterns/*.go`	Domain Patterns	Factory Method, Builder, Strategy, Observer (AML + velocity), and validation errors.
`internal/db/singleton.go`	Persistence	Thread-safe JSON ledger store (`LedgerStore`) with credit limit checks and atomic writes.
`internal/models/data.go`	Models	Serializable structs: `AccountData` , `LoanData` , `TransactionRecord` .
`web/index.html`	Frontend SPA	Hash-routed UI for login, dashboard, accounts, transfers, loans, and interest calculator.

2.1 REST API Endpoints

Method	Endpoint	Description	Patterns Involved
`GET`	`/api/accounts`	Returns all accounts sorted by creation time.	Facade, Singleton
`POST`	`/api/account`	Validates ID format and opens checking, savings, or credit account.	Facade, Factory Method, Singleton
`GET`	`/api/transactions`	Returns up to 100 latest ledger entries across accounts.	Facade, Singleton
`POST`	`/api/transaction`	Checks funds/credit limit/caps, runs fraud monitoring, and debits balance.	Facade, Observer, Singleton
`POST`	`/api/loan`	Validates borrower & policy bounds and generates a loan agreement.	Facade, Builder, Singleton
`GET`	`/api/interest`	Calculates annual interest for checking, savings, or credit/loan.	Facade, Strategy

3. Design Patterns Implementation

3.1 Singleton (`internal/db/singleton.go`)

Echo handles incoming HTTP requests in parallel goroutines, so direct file access from handlers would risk race conditions and lost updates. `LedgerStore` uses `sync.Once` (`initOnce`) for single-instance initialization and a `sync.RWMutex` around every read and write.

`writeLedgerFile()` serializes state to a `.tmp` file and performs an atomic `os.Rename` . `ApplyDebit()` enforces overdraft rules for checking/savings accounts and the $5,000 credit line (`DefaultCreditLimit`) for credit accounts. The file path can be overridden with `BANK_DB_PATH` so tests remain isolated.

```
func GetDBInstance() *LedgerStore {
        initOnce.Do(func() {
                path := os.Getenv("BANK_DB_PATH")
                if path == "" {
                        path = defaultLedgerPath
                }
                sharedLedger = openLedger(path)
        })
        return sharedLedger
}
```

3.2 Factory Method (`internal/patterns/factory.go`)

Account creation and account policies are decoupled through `Account` and `AccountCreator`. Each concrete type defines its own credit limit and single-transfer limit (Checking: $0 credit / $50,000 max transfer; Savings: $0 credit / $25,000 max transfer; Credit: $5,000 credit line / $5,000 max transfer). `CreateAccount` validates the account number format (2 to 24 alphanumeric, hyphen, or underscore characters) before delegating to `accountRegistry`.

```
var accountRegistry = map[string]AccountCreator{
        "checking": CheckingCreator{},
        "savings":  SavingsCreator{},
        "credit":   CreditCreator{},
}

func CreateAccount(accType string, id string) (Account, error) {
        id = strings.TrimSpace(id)
        if id == "" {
                return nil, fmt.Errorf("%w: account number is required", ErrInvalidInput)
        }
        if !validAccountID(id) {
                return nil, fmt.Errorf("%w: account number must be 2-24 chars (letters, digits, - or _)",
ErrInvalidInput)
        }
        creator, ok := accountRegistry[accType]
        if !ok {
                return nil, fmt.Errorf("%w: unknown account type %q", ErrInvalidInput, accType)
        }
        return creator.CreateAccount(id), nil
}
```

3.3 Builder (`internal/patterns/builder.go`)

Loan agreements combine borrower identity, principal bounds ($100 to $500,000), rate and term limits (up to 45% APR, 3 to 120 months), annuity math, and generated contract IDs. `StandardLoanBuilder` populates a draft step by step, `LoanDirector.ConstructStandardLoan` applies standard 12-month 15% terms, and `Build()` enforces underwriting policy before computing payments.

```
func (b *StandardLoanBuilder) Build() (*LoanAgreement, error) {
        agreement := *b.draft
        if agreement.Borrower == "" {
                return nil, fmt.Errorf("%w: borrower name is required", ErrInvalidInput)
        }
        if len([]rune(agreement.Borrower)) < 2 {
                return nil, fmt.Errorf("%w: borrower name is too short", ErrInvalidInput)
        }
        if agreement.Amount < MinLoanAmount || agreement.Amount > MaxLoanAmount {
                return nil, fmt.Errorf("%w: loan amount must be between $%.0f and $%.0f", ErrInvalidInput,
MinLoanAmount, MaxLoanAmount)
        }
        // monthly annuity formula: P * r / (1 - (1+r)^-n)
        monthlyRate := agreement.Rate / 100 / 12
        payment := agreement.Amount * monthlyRate / (1 - math.Pow(1+monthlyRate, -
float64(agreement.Months)))
        agreement.Monthly = math.Round(payment*100) / 100
        agreement.Total = math.Round(payment*float64(agreement.Months)*100) / 100
        agreement.Date = time.Now()
        agreement.Number = fmt.Sprintf("LN-%06d", agreement.Date.UnixNano()/1000%1000000)
        return &agreement, nil
}
```

For a standard loan of **$5,000** over 12 months at 15% APR, the monthly payment is **$451.29** and total repayment is **$5,415.50**.

3.4 Strategy (`internal/patterns/strategy.go`)

Annual yield calculation varies by account type through `InterestStrategy`: `CheckingStrategy` (0.0%), `SavingsStrategy` (5.0%), and `LoanStrategy` (15.0%, mapped to both `"credit"` and `"loan"`). `InterestContext` executes the active strategy selected by `GetStrategy(name)`.

`StandardLoanRatePercent = 15.0` is shared with `StandardLoanBuilder` so calculator rates and loan contracts stay synchronized.

3.5 Observer (`internal/patterns/observer.go`)

Every completed transfer triggers multiple side effects without coupling transfer execution to logging or alerting. `TransactionMonitor` notifies registered `Observer` instances sequentially:

- `FraudDetector` evaluates both the **$10,000 AML threshold** and **transfer velocity** (more than 3 transfers on the same account inside 60 seconds), populating `e.Suspicious` and `e.Reason`.
- `AuditLogger` prints a timestamped line with `OK` or `FLAGGED` status.
- `SecurityTeamAlert` logs an alert with the exact detection reason whenever `e.Suspicious` is true.

> **Registration order matters:** `BankFacade` registers `FraudDetector` before `SecurityTeamAlert` so the detector populates `Suspicious` and `Reason` before downstream observers act.

3.6 Facade (`internal/service/facade.go`)

`BankFacade` hides storage access, account policy checks, observer pipelines, and builders behind focused methods called by Echo routes. In `Transfer`, it validates funds, credit limits, and single-transfer caps before calling `txMonitor.ProcessTransaction`, ensuring rejected operations never pollute audit logs.

```
ev := f.txMonitor.ProcessTransaction(amount, accID)
entry := models.TransactionRecord{
        AccountID:  accID,
        Name:       "Transfer",
        Amount:     amount,
        Suspicious: ev.Suspicious,
        Reason:     ev.Reason,
        Date:       ev.Time,
}
balance, err := ledger.ApplyDebit(entry)
```

3.7 How the Patterns Work Together

Scenario	Execution Flow Across Patterns
Open Account	`Facade.OpenAccount` → **Factory Method** validates ID & sets credit limit → **Singleton** (`LedgerStore`) saves record.
Transfer	`Facade.Transfer` → **Singleton** checks funds/credit limit + **Factory** checks transfer cap → **Observer** (AML + velocity + audit + alert) → **Singleton** applies debit atomically.
Apply for Loan	`Facade.ApplyForLoan` → `LoanDirector` drives **Builder** (underwriting bounds + annuity) → saved via **Singleton**.
Calculate Interest	`Facade.CalculateYield` → **Strategy** selected by account type → executed in `InterestContext` .

4. Division of Work

The project was built by two team members, each implementing three design patterns alongside supporting infrastructure and tests.

Student	Patterns	Pattern Files	Additional Work
Student A: Shynarbek Nuraziz	Singleton, Factory Method, Facade	`internal/db/singleton.go` `internal/patterns/factory.go` `internal/service/facade.go`	Data models (`models/data.go`), HTTP API (`cmd/api/main.go`), DB and facade tests
Student B: Asset Kamza	Builder, Strategy, Observer	`internal/patterns/builder.go` `internal/patterns/strategy.go` `internal/patterns/observer.go`	Web frontend (`web/index.html`), Pattern unit tests, Project report & documentation

5. Clean Code Principles Applied

Principle	How It Is Applied	Where
Single Responsibility	Handlers parse HTTP and map codes; the facade enforces flow order; pattern files hold domain rules; `LedgerStore` only handles storage.	`main.go`, `facade.go`, `patterns/`, `singleton.go`
Open/Closed	New account types, interest schemes, or observers plug into registry maps or `AddObserver` without modifying callers.	`accountRegistry`, `interestStrategies`, `NewBankFacade`
Program to Interfaces	High-level logic interacts with `Account`, `AccountCreator`, `InterestStrategy`, `Observer`, and `LoanBuilder`.	`internal/patterns/`
DRY	Shared `StandardLoanRatePercent` constant across Builder and Strategy; shared HTTP helpers `writeErr`, `parseFloatParam`, `api`, `post`.	`strategy.go`, `main.go`, `index.html`
No Magic Numbers	Domain limits use named constants: `FraudThreshold`, `VelocityMaxInMin`, `DefaultCreditLimit`, `MinLoanAmount`, `MaxLoanAmount`.	`observer.go`, `singleton.go`, `builder.go`
Meaningful Names	Domain-specific symbols communicate intent: `LedgerStore`, `ApplyDebit`, `FraudDetector`, `ErrCreditLimitExceeded`.	Entire codebase
Fail Fast & Explicit Errors	ID formats, bounds, and caps fail immediately at boundaries; sentinel errors wrap with `%w`; `httpCodeFor` maps errors to HTTP statuses.	`patterns/`, `facade.go`, `main.go`
Encapsulation & Thread Safety	`LedgerStore.data` and `FraudDetector.recentHits` are protected by mutexes under concurrent requests.	`singleton.go`, `observer.go`
Consistency	Atomic `.tmp` rename writes prevent truncated JSON files; failed saves clean up in-memory state.	`builder.go`, `singleton.go`
Testability	`BANK_DB_PATH` and `openLedger(path)` isolate tests into temporary directories.	`singleton.go`, `*_test.go`

6. Testing & Verification

Automated tests cover domain rules, credit limits, velocity fraud detection, concurrent debits, and full facade workflows.

Test File	Test Functions	Verification Coverage
`internal/patterns/patterns_test.go`	`TestStrategySelection` `TestBuilderValidationAndPayments` `TestFactoryMethod` `TestObserverNotifiesAndDetectsVelocity`	0%/5%/15% interest rates, loan min/max/borrower checks and $451.29 payment, account ID validation, AML + velocity fraud triggers, unsubscription.
`internal/db/db_test.go`	`TestSeedAndPersistence` `TestDuplicateAccount` `TestDebitRulesAndCreditLimit` `TestConcurrentDebits`	Starter accounts and reload persistence, duplicate account error, overdraft protection, $5,000 credit limit enforcement, 50 concurrent debits.
`internal/service/facade_test.go`	`TestTransferFlow` `TestAccountsAndLoansAndInterest`	End-to-end scenarios through `BankFacade`, suspicious flagging with reasons, credit limit enforcement, newest-first transaction history.

6.1 Manual Demo Walkthrough

1. Opened credit account `CR-1` on Accounts page; repeating `CR-1` returned `an account with this number already exists`.
2. Transferred **$500** from `ACC-1` ($48,250.75 → $47,750.75) and logged an `[AUDIT] ... \| OK` line.
3. Transferred **$15,000** from `ACC-1`; flagged as **Suspicious**, displayed in Security Monitor, and printed `[ALERT]` with AML reason.
4. Attempted to transfer **$99,999** from `ACC-2`; rejected before Observer execution with `insufficient funds` (HTTP 422).
5. Generated a standard **$5,000** loan for Asset Kamza: 15% APR, 12 months, monthly payment **$451.29**, total **$5,415.50**.
6. Calculated savings interest on **$1,000**: annual yield **$50.00** (5%).

6.2 Automated Test Run (`go test -v ./... -count=1`)

```
?       ESHKEREE/cmd/api        [no test files]
=== RUN   TestSeedAndPersistence
--- PASS: TestSeedAndPersistence (0.16s)
=== RUN   TestDuplicateAccount
--- PASS: TestDuplicateAccount (0.00s)
=== RUN   TestDebitRulesAndCreditLimit
--- PASS: TestDebitRulesAndCreditLimit (0.01s)
=== RUN   TestConcurrentDebits
--- PASS: TestConcurrentDebits (0.09s)
PASS
ok      ESHKEREE/internal/db    1.166s
?       ESHKEREE/internal/models        [no test files]
=== RUN   TestStrategySelection
--- PASS: TestStrategySelection (0.00s)
=== RUN   TestBuilderValidationAndPayments
--- PASS: TestBuilderValidationAndPayments (0.00s)
=== RUN   TestFactoryMethod
--- PASS: TestFactoryMethod (0.00s)
=== RUN   TestObserverNotifiesAndDetectsVelocity
--- PASS: TestObserverNotifiesAndDetectsVelocity (0.00s)
PASS
ok      ESHKEREE/internal/patterns      0.790s
=== RUN   TestTransferFlow
[AUDIT] 23:17:04 | account ACC-1 | $500.00 | OK
[AUDIT] 23:17:04 | account ACC-1 | $15000.00 | FLAGGED
[ALERT] Suspicious transaction $15000.00 from account ACC-1 (exceeds $10000 AML limit)! Security team notified.
--- PASS: TestTransferFlow (0.15s)
=== RUN   TestAccountsAndLoansAndInterest
[AUDIT] 23:17:04 | account CR-9 | $300.00 | OK
--- PASS: TestAccountsAndLoansAndInterest (0.00s)
PASS
ok      ESHKEREE/internal/service       1.016s
```

6.3 Live Server Console Output (Audit & Security Alerts)

During the live browser demo walkthrough ($500 transfer followed by $15,000 transfer from `ACC-1`), the backend logged the observer pipeline outputs below:

```
____    __
  / __/___/ /  ___
 / _// __/ _ \/ _ \
/___/\__/_//_/\___/ v4.16.3
High performance, minimalist Go web framework
https://echo.labstack.com
_____________________________________O/_______
                                    O\
⇨ http server started on [::]:8080
[AUDIT] 23:27:34 | account ACC-1 | $500.00 | OK
[AUDIT] 23:27:35 | account ACC-1 | $15000.00 | FLAGGED
[ALERT] Suspicious transaction $15000.00 from account ACC-1 (exceeds $10000 AML limit)! Security team notified.
```

7. Application Screenshots

Screenshots captured directly from the running application in Chromium while executing the verification scenario above.

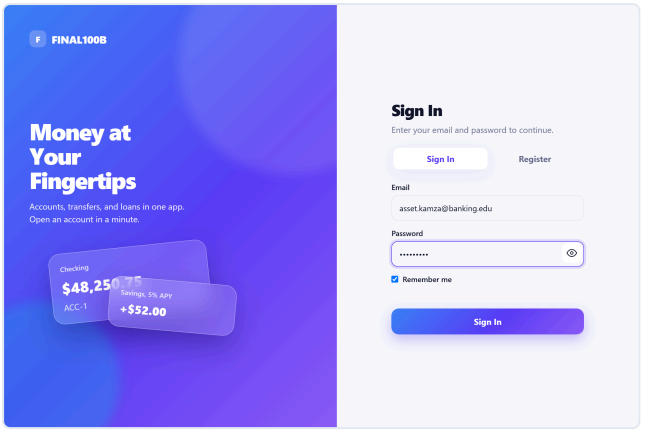


Figure 1. Sign-in page simulation

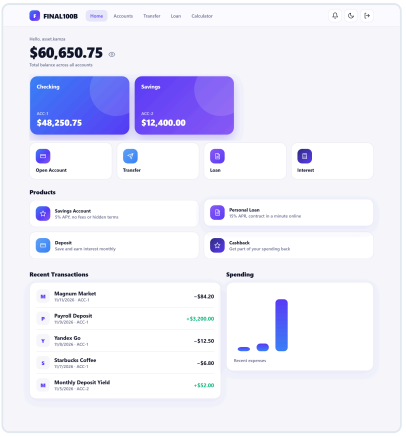


Figure 2. Home dashboard, cards, and spending chart

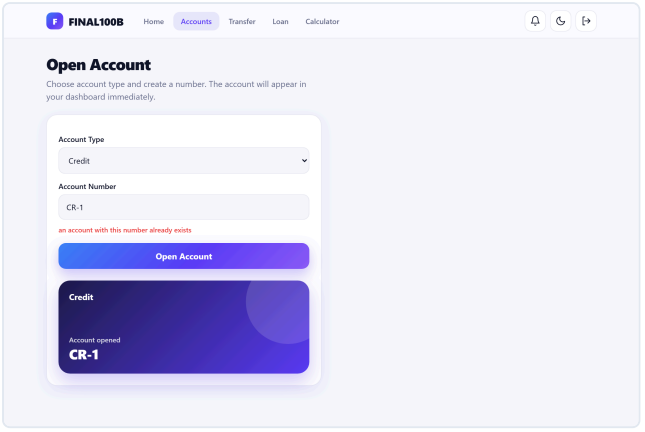


Figure 3. Accounts page: CR-1 opened and duplicate ID error

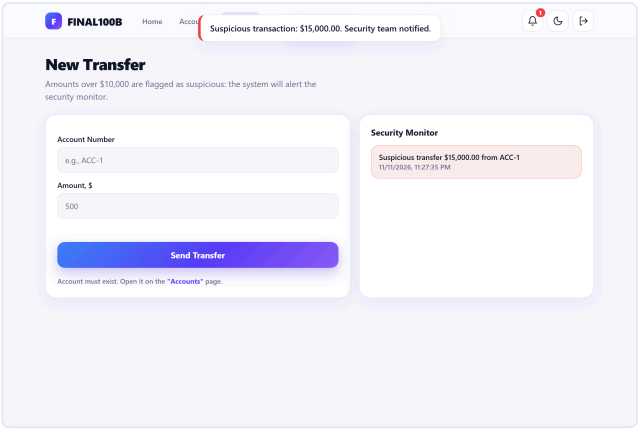


Figure 4. Transfer page: suspicious alert & Security Monitor

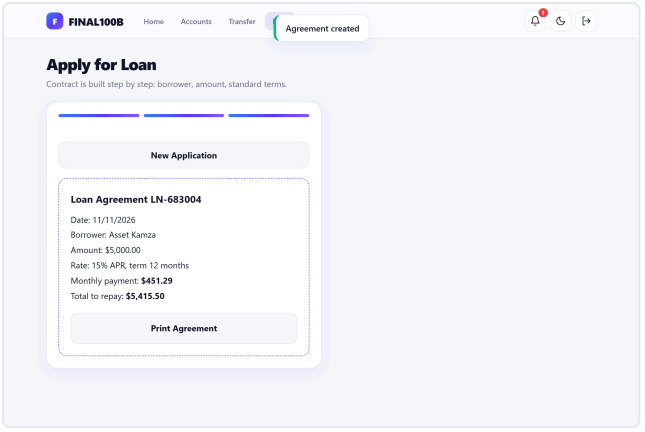


Figure 5. Loan page: generated agreement ($451.29/mo)

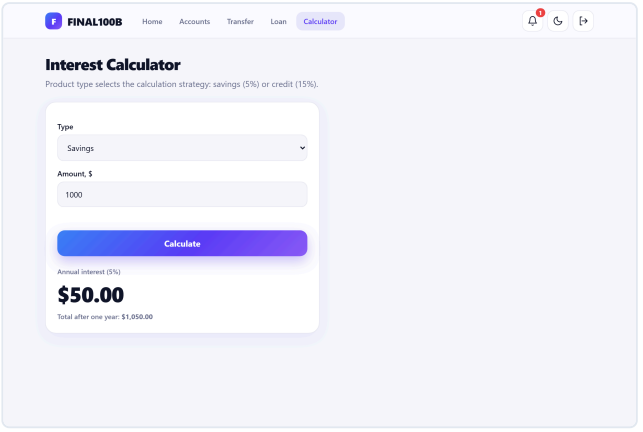


Figure 6. Interest Calculator: 5% savings result ($50.00)

8. UML Diagrams

Figure 7. Overall Class Diagram of the Six Patterns

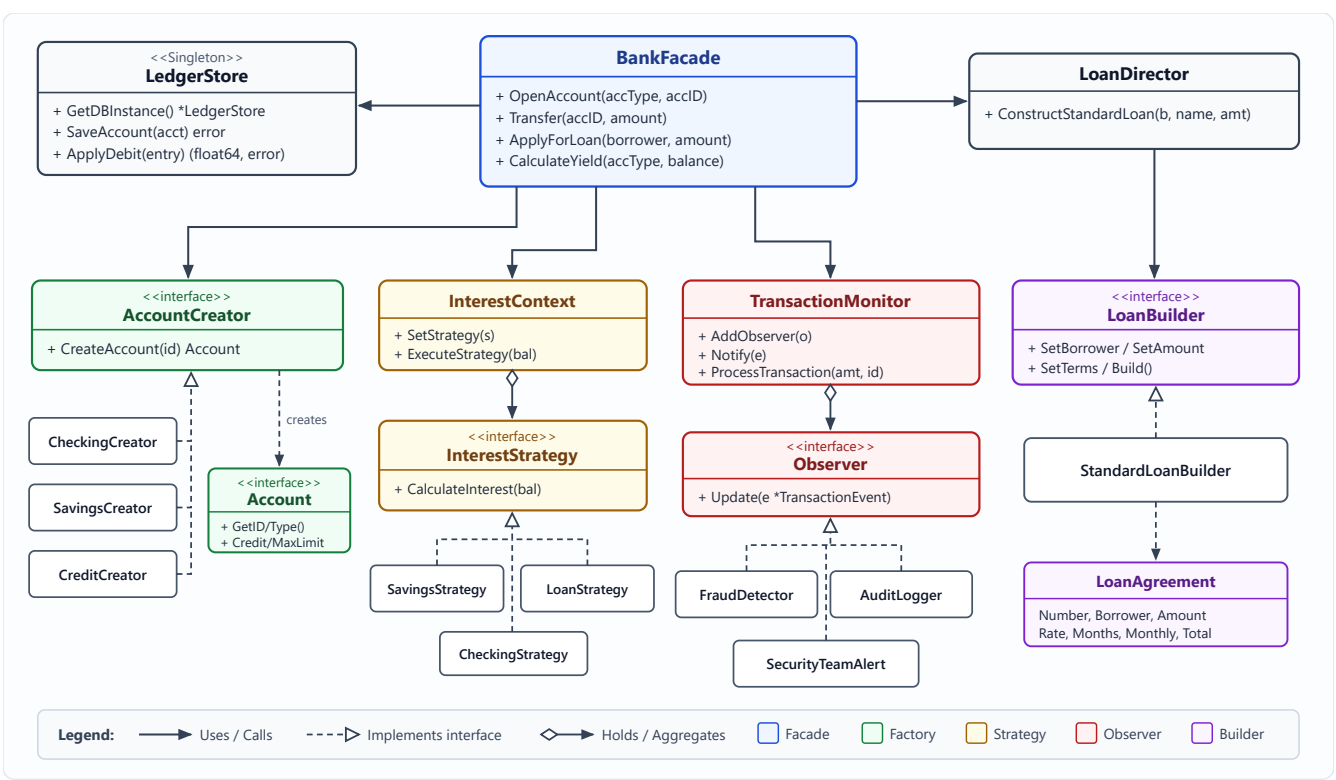
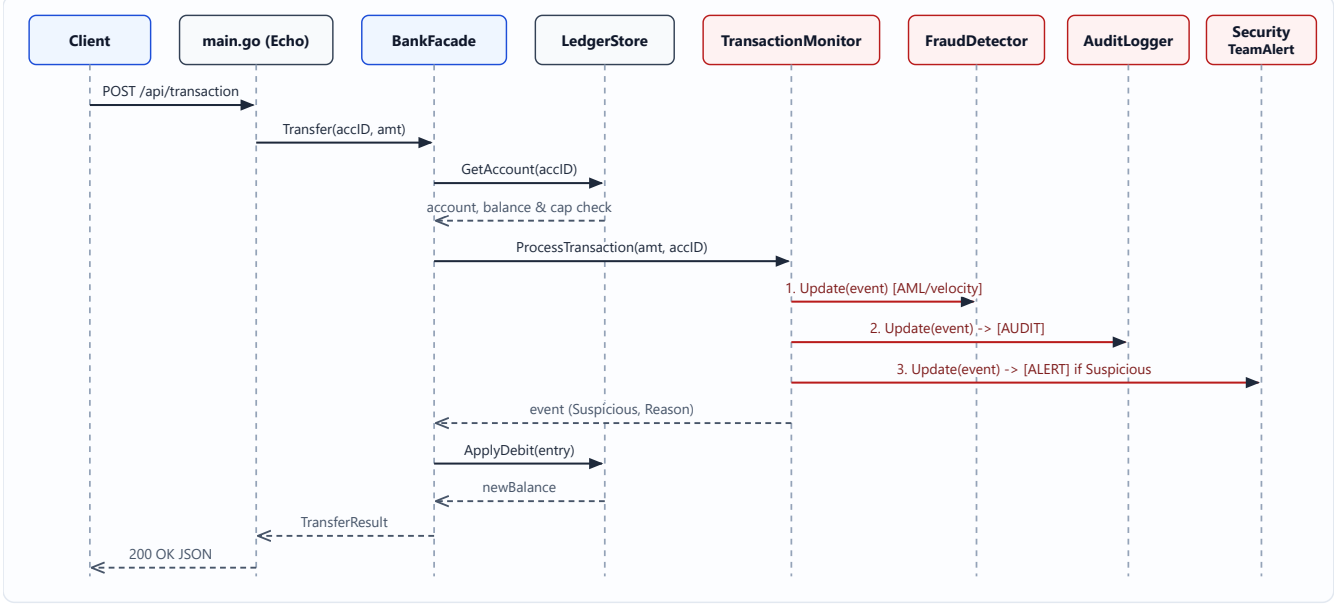


Figure 8. Sequence Diagram: Transfer Flow (`POST /api/transaction`)



9. Conclusion, Limitations & Further Work

We built a working banking web application in Go where six design patterns (Singleton, Factory Method, Builder, Strategy, Observer, and Facade) cooperate behind one service interface. Business constraints including account ID validation, per-product transfer limits, $5,000 credit limits, loan bounds, and AML + velocity fraud checks execute on the server, concurrent writes are synchronized under mutex locks, and 10 test functions verify storage persistence, pattern behavior, and facade workflows. Work was divided evenly between two students, three patterns each.

Current Limitations:

- Authentication on the frontend is simulated in browser `localStorage`; all visitors share the same server database file.
- Persistence relies on a local JSON file rather than a relational database.
- Generated loan agreements are stored separately and are not yet credited to or deducted from account balances.

Further Work:

- Add JWT-based server authentication with per-user account ownership.
- Add a **Decorator** pattern as a seventh pattern for transfer fees and extra compliance checks.
- Replace the JSON store with PostgreSQL behind a database interface without changing `BankFacade`.
- Support deposits, incoming transfers, and scheduled loan repayments from an account.
- Expand observers with email/SMS alerts, plus Docker and automated end-to-end tests.

10. References

1. Gamma E., Helm R., Johnson R., Vlissides J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
2. Freeman E., Robson E. *Head First Design Patterns*. O'Reilly Media, 2020.
3. Martin R. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
4. Refactoring.Guru. *Design Patterns Catalog*. https://refactoring.guru/design-patterns
5. The Go Programming Language Documentation. https://go.dev/doc
6. Echo Web Framework Documentation. https://echo.labstack.com

Appendix A. PlantUML Source Code

A.1 Overall Class Diagram

```
@startuml
title Banking System: Overall Class Diagram
class BankFacade {
  +OpenAccount(accType, accID)
  +Transfer(accID, amount)
  +ApplyForLoan(borrower, amount)
  +CalculateYield(accType, balance)
}
class LedgerStore <<Singleton>> {
  +GetDBInstance() : *LedgerStore
  +SaveAccount(acct) : error
  +ApplyDebit(entry) : (float64, error)
}
interface AccountCreator { +CreateAccount(id) : Account }
interface Account
class CheckingCreator
class SavingsCreator
class CreditCreator
interface InterestStrategy { +CalculateInterest(balance) }
class CheckingStrategy
class SavingsStrategy
class LoanStrategy
class InterestContext
interface LoanBuilder { +Build() : (*LoanAgreement, error) }
class StandardLoanBuilder
class LoanDirector
interface Observer { +Update(e *TransactionEvent) }
class TransactionMonitor
class FraudDetector
class AuditLogger
class SecurityTeamAlert
AccountCreator <|.. CheckingCreator
AccountCreator <|.. SavingsCreator
AccountCreator <|.. CreditCreator
AccountCreator ..> Account : creates
InterestStrategy <|.. CheckingStrategy
InterestStrategy <|.. SavingsStrategy
InterestStrategy <|.. LoanStrategy
InterestContext o--> InterestStrategy
LoanBuilder <|.. StandardLoanBuilder
LoanDirector --> LoanBuilder
Observer <|.. FraudDetector
Observer <|.. AuditLogger
Observer <|.. SecurityTeamAlert
TransactionMonitor o--> Observer
BankFacade --> LedgerStore
BankFacade --> AccountCreator
BankFacade --> InterestContext
BankFacade --> LoanDirector
BankFacade --> TransactionMonitor
@enduml
```

A.2 Sequence Diagram: Transfer

```
@startuml
title Transfer: POST /api/transaction
actor Client
participant "main.go (Echo)" as API
participant BankFacade as F
participant LedgerStore as DB
participant TransactionMonitor as M
participant FraudDetector as FD
participant AuditLogger as AL
participant SecurityTeamAlert as ST
Client -> API : POST /api/transaction (account_id, amount)
API -> F : Transfer(accID, amount)
F -> DB : GetAccount(accID)
DB --> F : account, balance & cap check
F -> M : ProcessTransaction(amount, accID)
M -> FD : 1. Update(event) [AML & velocity]
M -> AL : 2. Update(event) -> [AUDIT]
M -> ST : 3. Update(event) -> [ALERT] if Suspicious
M --> F : event (Suspicious, Reason)
F -> DB : ApplyDebit(entry)
DB --> F : newBalance
F --> API : TransferResult
API --> Client : 200 OK JSON (status, suspicious, balance)
@enduml
```